PROBLEM STATEMENT 8

Detection and Characterization of Subsurface Ice in Lunar South Polar Regions Using Chandrayaan-2 Radar and Imagery Data for Landing Site and Rover Traverse Planning

hackathon problem statement

The Big Picture (The "Elevator Pitch")

ISRO wants to find hidden water-ice on the Moon’s South Pole. Your job is to take raw satellite data from Chandrayaan-2, use it to prove exactly where the ice is buried inside a specific dark crater, and then design a safe roadmap for a future rover to land nearby and drive right up to that ice.

What You Actually Need to Do (Step-by-Step)

Think of this as a 4-part mission. During the 30 hours, you are going to act as a data scientist, a mission commander, and a navigator.

1. Find the Ice (Data Analysis)

- **The Problem:** The bottom of these craters are permanently dark ("doubly shadowed"). You can't just take a picture to see the ice.
- **Your Task:** You will be given radar data (DFSAR). Radar can bounce off the ground and tell you what's underneath. You need to write a script/workflow that analyzes these radar bounces to tell the difference between buried ice and just a bunch of rough rocks.

2. Pick a Landing Spot (Mission Planning)

- **The Problem:** You can't land a spacecraft inside a dark, jagged crater.
- **Your Task:** Look at the maps and elevation data (DEM) around the outside of the crater. Find a safe, flat spot that gets enough sunlight (for solar power) and isn't too far from the ice you just found.

3. Draw the Rover Path (Pathfinding/Navigation)

- **The Problem:** Rovers get stuck easily. They can't drive over massive boulders, go down steep cliffs, or survive too long in pitch-black darkness without their batteries dying.
- **Your Task:** Create an algorithm or use mapping tools to draw a physical "driving route" from your landing spot down into the crater to reach the ice. It has to be the safest and most energy-efficient path possible.

4. Do the Math (Estimation)

- **The Problem:** ISRO needs to know if this spot is worth the billions of dollars it costs to go there.
- **Your Task:** Use your radar analysis to calculate a rough estimate of exactly *how much* ice is buried in the top 5 meters of dirt in that area.

What Exactly Are You "Building"?

You are **not** building a software app, a website, or a physical robot.

You are building a **data pipeline and an analytical report**. Your final submission will likely be a presentation or a document backed by code, containing:

- **Maps:** Visual maps created in tools like QGIS/ArcGIS or Python showing where you think the ice is.
- **Coordinates:** The exact GPS-style lunar coordinates of your proposed landing site.
- **A Plotted Route:** A visual line drawn on a lunar map showing the rover's exact driving path, along with proof of *why* it's safe (e.g., "We chose this path because the slope is never more than 10 degrees").
- **A Number:** Your calculated estimate of the total ice volume.
- **Your Methodology:** The code (Python/GDAL/Rasterio) or workflows you used to process the Chandrayaan-2 radar data so they know you didn't just guess.

In short: They are giving you raw moon data. You need to build the analytical workflow that turns that raw data into an actionable treasure map for a future lunar rover.

- ***Where exactly is subsurface ice located?***
- ***How can ice be distinguished from rocky terrain?***
- ***Which sites are safe for landing?***
- ***How can a rover reach the target efficiently?***
- ***How much ice may actually exist?***

Datasets Required

- Chandrayaan-2 Dual Frequency Synthetic Aperture Radar (**DFSAR)**
- Chandrayaan-2 Orbiter High Resolution Camera (**OHRC**)

Quick Cheat Sheet:

- **OHRC** = "Above-ground" tasks (Slopes, craters, landing, driving, sunlight).
- **DFSAR** = "Underground" tasks (Finding ice, filtering rocks, calculating volume).

How to Use OHRC and DFSAR for the 4 Mission Phases

Here is the exact breakdown of which dataset you will use to solve each part of the problem statement.

Part 1: **Find the Ice (Radar Processing & Ice Mapping)**

- **Primary Dataset: DFSAR (Radar)**
- **Secondary Dataset: OHRC (Camera)**
- **How you use them:** You will use **OHRC** images to figure out the exact boundary of the pitch-black "doubly shadowed crater". Then, you will run your math formulas (calculating CPR and DOP) exclusively on the **DFSAR** data inside that boundary to locate the hidden ice and filter out the rocks.

Part 2: **Pick a Landing Spot (Terrain Analysis)**

- **Primary Dataset: OHRC (Camera & Elevation Data)**
- **How you use it:** You will not use radar for this. You will use the **OHRC** Digital Elevation Models (DEMs) to calculate the slope of the ground outside the crater. You will use the visual OHRC imagery to spot dangerous boulder fields and identify areas that get enough sunlight for the lander's solar panels.

Part 3: **Draw the Rover Path (Navigation)**

- **Primary Dataset: OHRC (Camera & Elevation Data)**
- **Secondary Input: The Ice Map (from Part 1)**
- **How you use them:** You need to draw a path from point A (the OHRC landing site) to point B (the DFSAR ice deposit). The actual "cost" of driving is calculated using **OHRC** elevation data. Your pathfinding algorithm will look at the OHRC DEMs to avoid steep crater walls and deep craters, ensuring the rover doesn't flip over while driving down to the ice.

Part 4: **Do the Math (Volumetric Ice Estimation)**

- **Primary Dataset: DFSAR (Radar)**
- **How you use it:** You will use pure physics based on the **DFSAR** radar signals. Radar strength (backscatter) changes depending on how much ice is mixed with the dirt. You will use the DFSAR intensity values within your detected ice zones to mathematically estimate how thick and dense the ice is in the top 5 meters.

dive right into Part 1: Finding the Ice (solution)

1The Science: Ice vs. Rocks (The Radar Ranges)

When the Chandrayaan-2 DFSAR shoots radar at the moon, the radar waves bounce back. We measure two things: **CPR** (Circular Polarization Ratio) and **DOP** (Degree of Polarization).

Both rocks and ice are highly reflective, but they reflect *differently*.

- **Rough Rocks (Boulder Fields):**
    - **How it bounces:** Radar hits the jagged, chaotic edges of rocks, bouncing violently in all random directions before returning to the satellite.
    - **Values: CPR > 1.0** (High brightness). Because the bouncing is chaotic, it scrambles the radar wave's original shape, resulting in a high **DOP (> 0.30)**.
- **Subsurface Water-Ice:**
    - **How it bounces:** The L-band radar (which penetrates ~5 meters deep) passes through the top layer of dry moon dust and hits the ice. The radar bounces around internally inside the clean, structured ice crystals (called *volumetric scattering*) and then escapes back to space.
    - **Values: CPR > 1.0** (High brightness). BUT, because ice is a clean, uniform dielectric medium, it *does not* scramble the radar wave. It preserves its structure, resulting in an ultra-low **DOP (< 0.13)**.

Your Golden Rule for the Hackathon Code: Formula :

IF (Pixel_CPR > 1.0) AND (Pixel_DOP < 0.13) = SUBSURFACE ICE

Mean : so, your core detection logic is a simple two-step filter: If the signal bounced a lot (CPR > 1.0) AND it didn't get scrambled (DOP < 0.13), you have found ice.

Algo use to training model for finding ice surface :

I highly recommend using **Support Vector Machines (SVM)** or **Random Forest** or **Fuzzy C-Means Clustering** (Unsupervised ML) or

CNNs are great but require heavy GPU resources, **Convolutional Neural Networks (CNN - PyTorch/TensorFlow).**

- *Why?* Instead of just looking at one pixel, a CNN looks at the *texture* of the surrounding pixels. Ice deposits often look like smooth "lobes" beneath craters, while rocks look like scattered noise. A VGG-16 or custom CNN architecture can detect these spatial patterns.

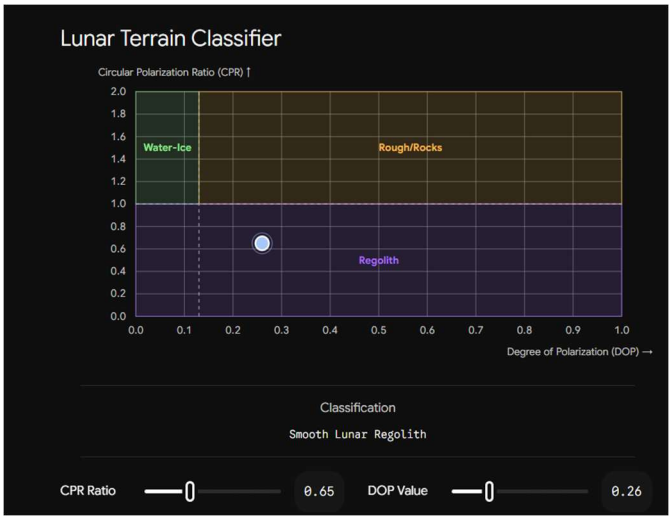

Image A – Rocks subSurface

Image B – Ice subSurface

The Logic: CPR and DOP Explained

Think of the radar signal as a perfectly organized spiral of energy being shot down at the Moon. When it hits the surface, it bounces back, and the satellite catches the echo.

1. CPR (Circular Polarization Ratio): *Did it bounce once or multiple times?*

CPR measures the physical roughness of the target. It looks at the ratio of the signal that returns in the *same* spiral direction versus the *opposite* spiral direction.

- **Low CPR (< 1.0):** The signal hit flat, smooth lunar dust (regolith) and bounced straight back once.
- **High CPR (> 1.0):** The signal hit something complex and bounced around multiple times before returning. Both **boulder fields** and **subsurface ice** cause this multiple-bounce effect, giving them both a high CPR.

2. DOP (Degree of Polarization): *Did the signal get scrambled?*

Because both rocks and ice trigger a high CPR, you need a second condition to filter out the false positives. DOP measures how "pure" or organized the returning signal is.

- **High DOP (> 0.13 to 0.35+):** The signal hit jagged, chaotic, rough rocks. The sharp edges scrambled the wave randomly, destroying its organized structure.
- **Low DOP (< 0.13):** The signal penetrated the dust and hit a clean, uniform sheet of ice. The wave bounced around internally inside the pure ice crystals (volumetric scattering) but *kept its organized structure intact*.

Expected Solution / Steps to be followed to achieve the objectives

- *Map permanently shadowed regions and identify doubly shadowed craters using illumination models and OHRC imagery.*
- *Analyze DFSAR data to compute radar parameters such as Circular Polarization Ratio (CPR) and Degree of Polarization (DOP). Apply refined criteria (e.g., CPR > 1 and DOP < 0.13) to identify potential subsurface ice signatures.*
- *Study crater morphology, slopes, boulder distribution, and surface roughness using OHRC data.*
- *Evaluate terrain safety and proximity to ice-bearing regions.*
- *Design an optimal and safe path considering terrain hazards and solar power constraints.*
- *Use radar backscatter models and dielectric assumptions to estimate ice concentration and volume within the top 5 meters.*

In this Challenge #8, we are asking you to think like mission planners. Your task is to move from orbital observations to an actionable exploration strategy: where to land, where to drive, where to excavate, and how much ice may be available.

Your challenge is to discover hidden lunar ice and design a realistic pathway for future ISRO missions to reach and utilize it.

Phase 2: Landing Site Selection (Terrain & Safety Analysis)

1. The Strategy (The Proximity Paradox) We cannot land directly on the ice because the target is a Permanently Shadowed Region (PSR) with extreme cold (-200°C) and zero solar energy. Instead, we must select a "Sunlit Basecamp" on the elevated crater rim. This provides a safe, powered landing zone with a feasible drive down into the crater.

2. The Three Engineering Constraints To validate a safe landing spot using OHRC (Orbiter High Resolution Camera) data, the site must pass three filters:

1. **Topography (Slope):** The surface slope must be **< 10°** to prevent the lander from tipping over upon touchdown.
2. **Illumination:** The site must receive **> 80%** continuous sunlight over the lunar day to ensure solar panel efficiency and thermal survival.
3. **Hazard Clearance:** The landing ellipse must be free of large boulders (taller than 30 cm) and deep micro-craters.

3. Tech Stack & ML Architecture

- **Geospatial Processing (Python):** Using rasterio and GDAL to process OHRC 3D Digital Elevation Models (DEMs) and calculate the mathematical slope/gradient of every pixel.
- **Hazard Detection (YOLO / U-Net):** Implementing computer vision models (like YOLO for object detection or U-Net for semantic segmentation) on OHRC optical imagery to automatically detect and map boulders and dangerous craters.
- **Suitability Mapping:** Combining the slope, illumination, and hazard maps using a multi-criteria GIS algorithm (or XGBoost scoring) to output a final 0-100 "Safety Score" for potential landing coordinates.

Earth Navigation vs. Lunar Navigation

Feature	Earth Pathfinding (e.g., Google Maps)	Lunar Pathfinding (Rover Navigation)
The Network	**Road-Based:** The algorithm is locked to a predefined vector network of roads and streets. It only has to choose *which turn* to take.	**Grid-Based (Open Terrain):** There are no roads. The algorithm looks at a continuous grid of pixels across open terrain and must evaluate every square meter.
The Infrastructure	**GPS Network:** Constant connection to dozens of global satellites that pinpoint exact locations within centimeters.	**No GPS:** The rover must calculate its position using wheels turning (dead reckoning), star tracking, and matching camera images to satellite maps.
Primary Penalty	**Time (Traffic):** The "cost" of a route is usually time. A red highway means slow speeds, forcing the algorithm to find a faster detour.	**Survival (Hazards & Power):** The "cost" is danger. A steep slope or a dark shadow isn't just a delay; it can permanently flip, crash, or freeze the rover.
Energy Constraints	**Ignored:** Gas or electric cars can easily pull over, refuel, or change paths without worrying about starving for energy mid-drive.	**Critical (Solar/Battery):** The path is strictly limited by how long the battery can last in the dark before returning to sunlight.